

Git Student Guide

AS-Level Computer Science

Version Control with Git and GitHub

1 What is Git?

Git is a **version control system**. It tracks changes to your code and lets you:

- Save your progress at any point (like checkpoints in a game)
- Go back to previous versions if something breaks
- Work with others on the same project
- Keep your code backed up online (on GitHub)

1.1 Git vs GitHub

Git	GitHub
Software on your computer	Website (github.com)
Tracks changes locally	Stores your code online
Works offline	Requires internet
The tool	The cloud storage

Think of Git as Microsoft Word's "Track Changes" feature, and GitHub as OneDrive where you store your documents.

2 Opening the Terminal

The terminal (also called command line) is where you type Git commands.

2.1 Windows

- Press **Windows** key, type **cmd** or **PowerShell**
- Or: Right-click in a folder and select "Open in Terminal"

2.2 Mac

- Press **Cmd + Space**, type **Terminal**
- Or: Go to Applications ↪ Utilities ↪ Terminal

3 Basic Terminal Commands

Before using Git, you need to know how to navigate folders in the terminal.

3.1 cd – Change Directory

Move into a folder:

Change Directory

```
cd Documents
cd my-project
cd ..
```

- `cd Documents` – Go into the Documents folder
- `cd my-project` – Go into a folder called my-project
- `cd ..` – Go back to the parent folder (one level up)

3.2 mkdir – Make Directory

Create a new folder:

Make Directory

```
mkdir my-new-folder
```

3.3 ls / dir – List Files

See what files are in the current folder:

List Files

```
ls          # Mac/Linux
dir         # Windows
```

3.4 pwd – Print Working Directory

See which folder you are currently in:

Print Working Directory

```
pwd          # Mac/Linux
cd           # Windows (just cd with no arguments)
```

4 Git Commands

4.1 git clone – Download a Repository

Download a project from GitHub to your computer:

Clone a Repository

```
git clone https://github.com/username/repository.git
```

This creates a new folder with all the project files. After cloning, move into the folder:

```
cd repository
```

4.2 git status – Check What Changed

See which files have been modified:

Check Status

```
git status
```

This shows:

- Files you have changed (in red)
- Files ready to be saved (in green)
- Which branch you are on

4.3 git add – Stage Changes

Tell Git which files you want to save:

Stage Changes

```
git add .                # Add ALL changed files
git add filename.py      # Add one specific file
```

The . means "everything in the current folder".

4.4 git commit – Save Changes

Save your staged changes with a message describing what you did:

Commit Changes

```
git commit -m "Added pathfinding algorithm"
```

The message should be short but descriptive. Good examples:

- "Fixed bug in neighbor detection"
- "Added Level 2 solution"
- "Completed A* implementation"

Bad examples:

- "stuff"
- "changes"
- "asdfgh"

4.5 git push – Upload to GitHub

Send your commits to GitHub:

Push to GitHub

```
git push origin main
git push origin your-branch-name
```

4.6 git pull – Download Updates

Get the latest changes from GitHub:

Pull from GitHub

```
git pull
```

Use this when your partner has pushed changes and you need to get them.

5 Working with Branches

A **branch** is your own copy of the code where you can make changes without affecting the main project.

5.1 Why Use Branches?

- Your work stays separate from others
- You can experiment without breaking the main code
- Easy to review changes before merging

5.2 git checkout -b – Create a New Branch

Create and switch to a new branch:

Create New Branch

```
git checkout -b team-alice-bob
```

Replace `team-alice-bob` with your team name.

5.3 git branch – See All Branches

List all branches (the * shows which one you're on):

List Branches

```
git branch
```

5.4 git checkout – Switch Branches

Switch to a different branch:

Switch Branch

```
git checkout main
git checkout team-alice-bob
```

6 The Complete Workflow

Here is the typical workflow for your project:

6.1 Step 1: Clone the Repository

Step 1

```
cd Documents
git clone https://github.com/teacher/pathfinding.git
cd pathfinding
```

6.2 Step 2: Create Your Branch

Step 2

```
git checkout -b team-yournames
```

6.3 Step 3: Do Your Work

Edit the files, write your code, test it.

6.4 Step 4: Save Your Work

Step 4

```
git add .
git commit -m "Describe what you did"
git push origin team-yournames
```

Repeat Step 3 and 4 every time you complete a working piece of code.

7 Common Problems and Solutions

7.1 Problem: "fatal: not a git repository"

You are not inside a Git folder. Use `cd` to navigate into your project folder first.

7.2 Problem: "Permission denied"

You need to log in to GitHub. Try:

```
git config --global user.name "Your Name"
git config --global user.email "your.email@example.com"
```

7.3 Problem: "Your branch is behind"

Someone else pushed changes. Pull first, then push:

```
git pull
git push
```

8 Git Cheat Sheet

Command	What it does
cd foldername	Go into a folder
cd ..	Go back one folder
mkdir foldername	Create a new folder
ls / dir	List files
git clone url	Download a repository
git status	See what changed
git add .	Stage all changes
git commit -m "msg"	Save with a message
git push	Upload to GitHub
git pull	Download updates
git checkout -b name	Create new branch
git branch	List all branches
git checkout name	Switch to branch

Remember

Commit often! Every time something works, save it with:

```
git add .
git commit -m "What you did"
git push
```